Gradient Descent

Linear Regression (Recap)

- Goal:
  - Given a training set, learn θ
  - Such that $h_\theta(x)$ is as close as possible to y

Least Mean Squares (LMS)

- For a dataset with n samples, minimize

$$J(\theta) = \frac{1}{2}\sum_{k=1}^{n}\left(h_\theta(x^{(k)}) - y^{(k)}\right)^2$$

Cost Function/Loss Function

$$h_\theta(x) = \sum_{i=0}^{d}\theta_i x_i = \theta^T x$$

Linear Regression

- Goal: Choose θ to minimize $\mathrm{J}(\theta)$

$$\theta^* = argmin_\theta J(\theta)$$

- How?

- Gradient Descent

Gradient Descent

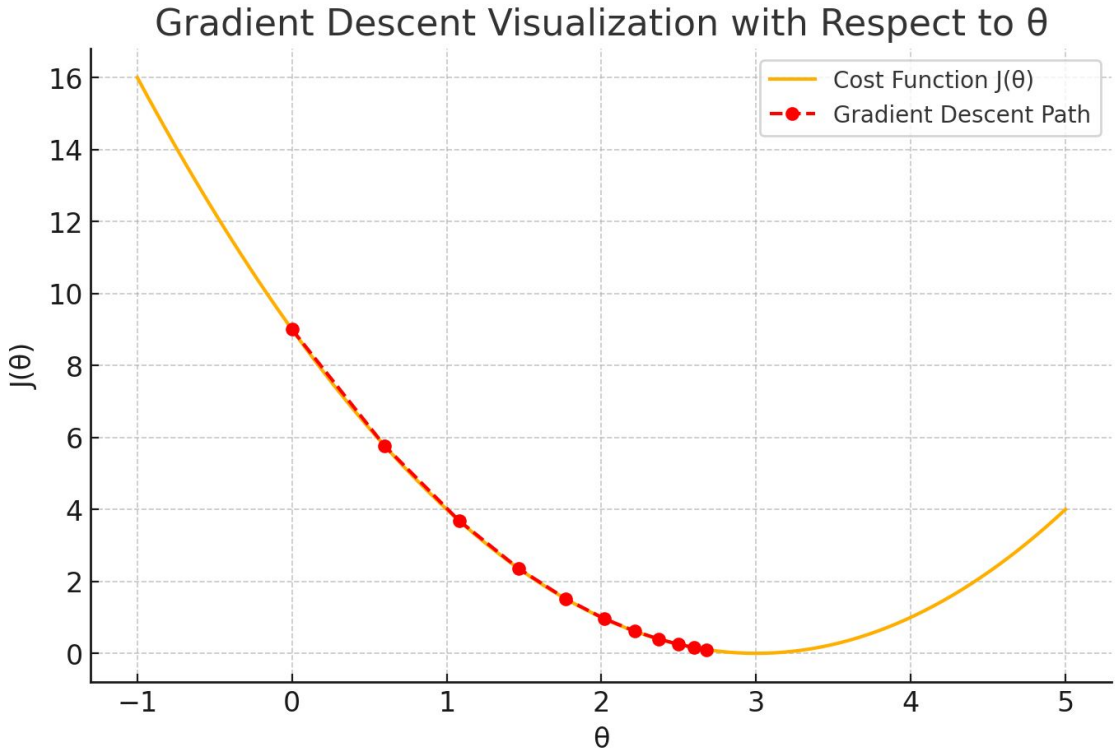
Cost Function

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}(h_\theta(x^{(i)}) - y^{(i)})^2$$

- Start from an initial θ
- Repeatedly update θ as follows

$$\theta_j := \theta_j - \boxed{\alpha \frac{\partial}{\partial \theta_j} J(\theta)}$$

- θ_j corresponds to the coefficient of the x_j
- α is called learning rate (hyper-parameter)
  - Constant
  - Decaying



Gradient Descent

$$\theta_j := \theta_j - \boxed{\alpha \frac{\partial}{\partial \theta_j} J(\theta)}$$

Cost Function

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)})^2$$

$$\begin{aligned}
\frac{\partial J}{\partial \theta_j} &= \frac{1}{2} \sum_{k=1}^{n} \frac{\partial}{\partial \theta_j} \left(h_\theta(x^{(k)}) - y^{(k)}\right)^2 \\
&= \frac{1}{2} \sum_{k=1}^{n} 2\left(h_\theta(x^{(k)}) - y^{(k)}\right) \frac{\partial}{\partial \theta_j} h_\theta(x^{(k)}) \\
&= \sum_{k=1}^{n} \left(h_\theta(x^{(k)}) - y^{(k)}\right) \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{d} \theta_i x_i^{(k)}\right) \\
&= \sum_{k=1}^{n} \left(h_\theta(x^{(k)}) - y^{(k)}\right) x_j^{(k)}
\end{aligned}$$

Gradient Descent

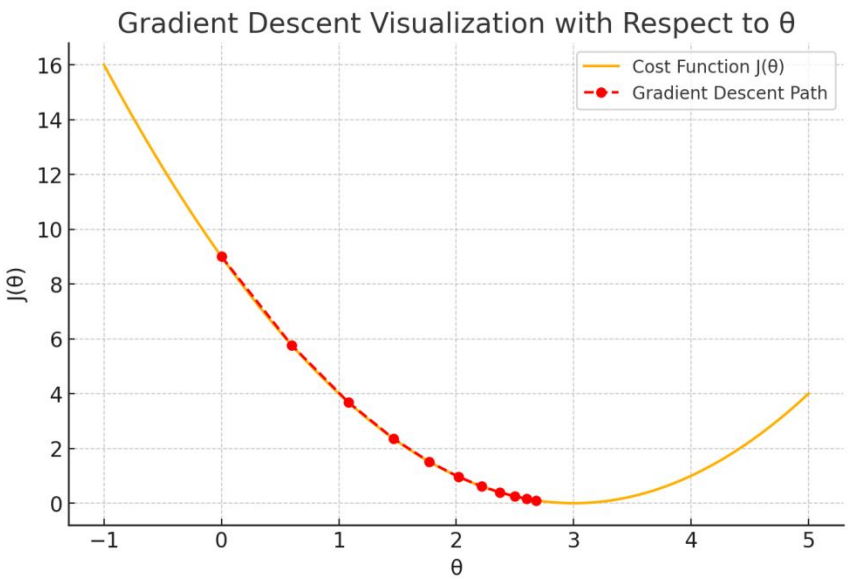
$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}(h_\theta(x^{(i)}) - y^{(i)})^2$$

Cost Function

- Start from an initial θ

Repeat until convergence

$$\theta_j := \theta_j + \alpha \sum_{k=1}^{n} \left(y^{(k)} - h_\theta(x^{(k)})\right) x_j^{(k)}, \quad \text{(for every } j\text{)}$$



The Three Flavors of Gradient Descent

- Batch Gradient Descent
  - Use all n samples to compute the gradient
  - Exact gradient
  - Stable Updates
  - Slow, memory intensive
- Stochastic Gradient Descent
  - Use 1 sample to compute the gradient
  - Very fast updates
  - Noisy gradient, Convergence issues
- Mini-batch Gradient Descent
  - Use a small batch of samples
  - Reasonably stable
  - Most used choice

The Three Flavors of Gradient Descent

Let, $J(\theta, x, y)$ is the loss from sample (x, y)

- Batch Gradient Descent

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{N} \sum_{i=1}^{N} J(\theta, x_i, y_i)$$

- Stochastic Gradient Descent

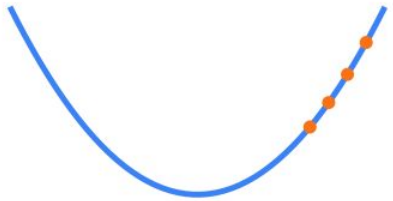
$$\theta_j \leftarrow \theta_j - \alpha \, J(\theta, x, y)$$

- Mini-batch Gradient Descent

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} J(\theta, x_i, y_i)$$

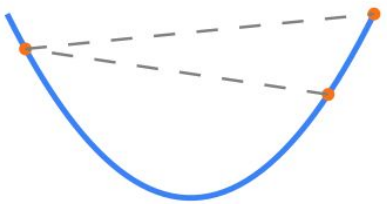
Learning Rate

- Too small



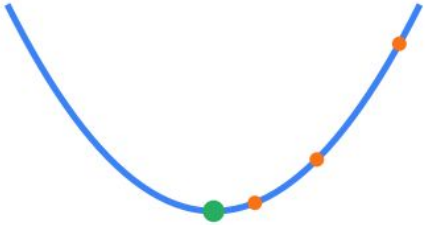
Painfully slow!

- Too Large



Oscillates/diverges!

- Just Right



Converges nicely!

Reference

- D2l
- Deep Learning